### *WHY REPOS AREN'T STORES*

When ACME first adopted policy as code, it was tempting to think, "We'll just put our Cedar files in Git." However, the identity team quickly realized that a policy repository and a policy store serve different purposes.

A *policy repository* is where policies are created, reviewed, and versioned. It's human-facing: Cedar files are stored alongside their schemas, test cases, and documentation. Changes are submitted via pull requests, automatically checked by the continuous integration process, and approved by code owners familiar with the business rules. The repository acts as the definitive source for how authorization logic should function.

A *policy store*, on the other hand, is what the Cedar authorizer queries during runtime. It doesn't hold drafts or pull requests. Instead, it stores signed, versioned policy bundles that have already passed review and testing. The store is designed for machine use, prioritizing reliability and auditability rather than authoring. It records decisions, enforces SLAs, and supports clean rollbacks if issues arise in production. In Amazon Verified Permissions, for example, a policy store keeps versioned, signed bundles generated from the repository, ensuring integrity from creation to deployment.

Early on, an ACME engineer attempted to hotfix a production policy by editing the store directly. The change lasted for a day, but no one could trace its origin, and the next deployment overwrote it. Switching to signed bundles prevented this kind of drift. From then on, the only way to modify a policy store was by way of a reviewed pull request and a new bundle, ensuring ACME both reliability and accountability.

Keeping repositories and stores separate gave ACME the best of both worlds: Product teams like Customer Collaboration could manage their own repositories—writing policies they understood best—while the platform team handled the corresponding stores that applications relied on.